



Qatar University

Internship Report

At Scale AI

Cohort Flow Simulator

Najeeb Barkhad (202307172)

Mentor: Dr Saleh AL Hazbi - Mr Ayman Adil

2026

This project report is submitted to the Department of Computer Science and Engineering of Qatar University in partial fulfillment of the requirements of the Practical Training course.

Abstract

During my internship, completed on Qatar University's campus with Scale AI as the remote host organization, I worked on the design and development of the Single-Cohort Flow Simulator, a software system built to study student progression through Qatar University's Computer Science curriculum. The project focuses on an academic planning problem: when students are delayed, it is often difficult to determine whether the delay was caused by course difficulty, missing prerequisites, limited seats, or courses being offered only in certain terms.

To address this problem, I developed a discrete-term, agent-based simulation model that represents students as individual agents moving through the curriculum over time. The system models prerequisites, course offerings, pass and fail outcomes, seat capacity, academic probation, dropout, and graduation. It has since grown from its original single-cohort design into a multi-cohort, steady-state model in which several overlapping cohorts are admitted year after year and compete for one shared pool of course seats, which is closer to how a real department actually operates. I also contributed to the FastAPI backend, SQLite persistence layer, and Next.js dashboard that make the simulator usable as an interactive decision-support tool.

The final result is a working platform that can run curriculum simulations, identify bottlenecks through separate failure, capacity, offering, and prerequisite signals, support what-if analysis through a scenario tester and a bounded capacity-planning solver, and present the results visually through a web dashboard. On the current baseline configuration, the simulator reports a steady-state graduation rate of about 66% within a 12-semester horizon, with mathematics gateway courses driving most failures and major-elective seats driving most capacity pressure; these findings are discussed in the Internship Experience section below. This internship strengthened my skills in simulation modeling, backend development, database design, frontend implementation, testing, and technical documentation.

Acknowledgment

I would like to express my sincere gratitude to my mentors and colleagues at Scale AI for their guidance and support throughout this internship, and to my instructors at Qatar University for their feedback on both the technical quality of the project and the way I explained its purpose.

I would also like to thank the Department of Computer Science and Engineering at Qatar University for providing the academic foundation, and the campus environment, that allowed me to work on a project combining software engineering, data analysis, and educational planning. This internship gave me the opportunity to apply concepts from my coursework to a practical system with real academic relevance, while working with a remote host organization.

Table of Contents

Abstract

Acknowledgment

1. Background of the organization
2. Internship learning objectives
3. Internship Experience
4. Reflections and learning outcomes

1. Background of the organization

The Host Organization: Scale AI

This internship was completed on Qatar University's campus, with Scale AI as the remote host organization, through the **Scale x Qatar University Practical Training program**, a joint arrangement in which Scale AI, a technology company that builds data-labeling, evaluation, and human-feedback infrastructure used by AI research labs and enterprises to train and evaluate machine learning models, hosts QU students on independent, self-directed technical projects. Students are assigned a task from a shared Practical Training Task Catalog: each task is scoped for one student to complete solo, end-to-end, in roughly 25-30 hours of work per week across the seven-week training window, and is graded against a common rubric worth 15% of the practical training grade, scored across five bands from Excellent to Failing. This is how this simulator came to be the deliverable: rather than working on Scale AI's own data pipelines, I built a data-driven decision-support tool for a curriculum-planning problem I had first-hand context on as a Qatar University Computer Science student. The engineering habits the internship emphasized, treating a system as something to be measured and evaluated with structured evidence rather than a single headline number, carried directly into how this project's own bottleneck analytics were designed, discussed later under Internship Experience.

The Academic Project: Qatar University's Computer Science Curriculum

Qatar University is the national university of Qatar and offers a Bachelor of Science program in Computer Science through the College of Engineering. The Computer Science curriculum includes a structured sequence of programming, mathematics, computer systems, software engineering, databases, networks, cybersecurity, electives, and senior project courses. Like many technical programs, this curriculum depends heavily on prerequisite chains and term-based course offerings.

The project developed during this internship, the Single-Cohort Flow Simulator, was designed to support academic planning for this kind of curriculum. Its name reflects the project's origin as a model of a single incoming class; it has since grown into a multi-cohort, steady-state model of the whole department, and the name was kept for continuity rather than renamed mid-project. The main idea behind the project is that student delay cannot be understood only by looking at final graduation numbers. A student may be delayed because a difficult course was failed, because an important course was full, because a prerequisite was not completed, or because a required course was not offered in the needed term. These causes look similar in a transcript, but they require different solutions.

The simulator therefore models student progression term by term. It uses the 2024 Qatar University Computer Science study plan as its curriculum structure, including 41 courses and 120 credit hours. Students are simulated as individual agents who attempt to register for courses, compete for limited seats, pass or fail courses, and move toward graduation. Rather than modeling one class in isolation, the engine now admits a new cohort every year and keeps several cohorts enrolled at once, all drawing from the same pool of course seats, so that a shortage in one gateway course is felt by freshmen and seniors alike. The system records the reason each student becomes blocked, allowing the project to identify curriculum bottlenecks in a more explainable way.

The final application includes a Python simulation engine, a FastAPI backend, a SQLite database layer, and a Next.js dashboard. These components work together to provide an interactive tool for running simulations, reviewing bottlenecks, testing policy changes, and viewing academic flow through charts and dashboard pages.

2. Internship learning objectives

Activity Objectives

At the beginning of the internship, the main activity objectives were:

- Understand the structure of the Qatar University Computer Science curriculum and identify the factors that can delay student progression.
- Build a simulation model that represents students, courses, prerequisites, terms, pass rates, and seat capacity.
- Develop backend services that expose simulation results through a clean API.
- Create a database model that stores plans, courses, configuration, scenarios, and run history.
- Build a web dashboard that presents simulation results in a clear and interactive way.
- Add analytics that separate different bottleneck causes instead of combining them into one score.
- Document the system so that future users and developers can understand the design decisions.

Growth Objectives

The personal growth objectives of the internship were:

- Improve my ability to work independently on a long-running software project, including while working with a remote host organization rather than on-site.
- Strengthen my understanding of backend architecture, API design, and database modeling.
- Gain more experience with React and TypeScript through a practical dashboard interface.
- Learn how to translate a real academic planning problem into a software model.
- Improve my debugging and testing skills, especially for deterministic simulation behavior.
- Become better at explaining technical work to both technical and non-technical audiences.

Significance of the Objectives

These objectives were important because they connected directly to my academic studies and future career goals. The project required knowledge from data structures, algorithms, database systems, software engineering, probability, and statistics. It also required practical decision-making, because the system had to be useful, understandable, and maintainable rather than only technically complex.

The internship helped me move from classroom-based understanding to applied development. Instead of building a small isolated assignment, I worked on a system with multiple layers, many design decisions, and a clear user purpose.

Connection to the Organizations' Missions

The project connects to two missions at once. It connects to Qatar University's mission by supporting academic quality, data-informed planning, and student success: by making curriculum bottlenecks visible, the simulator can help academic planners discuss possible improvements to course capacity, prerequisite structure, offering patterns, and admission size. It also reflects the engineering discipline emphasized at Scale AI, where rigorous, structured evaluation of a complex system is the core of the business: this project applies that same instinct to a university curriculum, replacing a single graduation-rate number with four separately tracked bottleneck signals so that a claim about "why students are delayed" is backed by specific, checkable evidence rather than intuition. The system does not replace academic judgment, but it provides structured evidence that can support better decisions.

3. Internship Experience

The assigned catalog task was to design and build, independently and end-to-end, a data-driven decision-support system for a real institutional planning problem; I chose Qatar University's own Computer Science curriculum as that problem, given my direct exposure to it as a QU CS student. The work progressed from building the core simulation engine, to turning it into a full-stack application, to iteratively adding and testing decision-support features against the research question and cutting or rebuilding whatever did not hold up, and finally to tightening the whole system and its documentation before submission.

Before walking through what was actually built, it is worth describing how the system works overall, since the account below refers back to it repeatedly.

Overall Approach

The project followed a research-driven software engineering approach. The first step was to define a clear problem: how to explain student delay in terms of curriculum structure rather than only final outcomes. Once the problem was understood, it was translated into a model where each student progresses through the curriculum over time according to a consistent set of academic rules and institutional constraints.

Simulation Model

The simulator uses a discrete-term loop with three phases repeated every academic term. In the first phase, each active student builds a priority-ordered list of desired courses: any failed course they must retake comes first, followed by courses grouped into configurable priority tiers (core and college-requirement courses first, electives only once a student has earned enough credit hours, then remaining math, science, English, and general-education requirements), subject to a course-load cap that is reduced further for students on academic probation. In the second phase, every student who requested a given course is ranked by class standing and a random tiebreak, and seats are granted in that order until the course's capacity is exhausted; everyone else is recorded as capacity-blocked. In the third phase, each granted enrollment resolves to a pass or fail outcome, and the student's GPA, completed credit hours, probation status, and graduation status are updated accordingly. Because seat allocation is ranked by class standing across the whole shared pool, senior students from older cohorts outrank incoming freshmen automatically, without any special-case code.

Students are not treated as a single average cohort; instead, they retain individual histories, completed credits, GPA, probation status, and progression patterns. A separate personal counter tracks how many mandatory (Fall/Spring) semesters a student has personally used, distinct from the university's overall calendar term; this distinction matters because the curriculum also includes optional Summer/Winter terms in which a student can take a service course without spending one of their limited semesters. This makes the model more realistic because students often follow different paths even when they belong to the same department or program, and it prevents the optional terms from silently distorting how long a student is judged to have taken.

Architectural Design

The implementation was organized using a layered architecture. The simulation engine is implemented in the core Python module, while the service layer exposes the engine through a structured interface with no file I/O of its own. The API layer provides HTTP access, the database layer handles persistence, and the frontend dashboard presents the results in a visual and interactive format. This separation of concerns was important because it allowed the project to grow from a script-based prototype into a more complete academic planning tool without mixing the responsibilities of the different layers, and it is what let the same simulation

engine be reused unchanged by a command-line script, an HTTP API, and a resumable, term-by-term checkpoint session (covered later in this section, when Semester Checkpoint Mode is discussed).

Analytics and Decision Support

The project also includes an analytics layer that transforms raw simulation output into useful planning information. Instead of reporting only final graduation rates, the system records four separate bottleneck signals every term: a *failure* count when a student attempts and fails a course, a *capacity* count when an eligible student requests a seat but loses the allocation, an *offering* count when an eligible student's course simply is not taught that term, and a *prerequisite* count when a student is still waiting on an upstream course. These four signals are never merged into a single score, which is what makes it possible to say whether a given course is a problem because it is hard, because it is oversubscribed, because it is scheduled too rarely, or because it is downstream of a different, harder gateway.

Course capacity itself is not sized uniformly: because a missed seat in an early, once-a-year gateway course can push a student off the normal annual rhythm and into a full year's delay, the required core sequence and all non-elective service courses are sized to peak historical demand, while the interchangeable major electives are deliberately squeezed to a lower percentile so that some contention remains where it does the least harm. This sizing policy, together with the block signals above, is what the results later in this section are read against.

Why This Method Was Appropriate

This methodology was appropriate because it balanced realism, explainability, and flexibility. The model captures the logic of academic progression while remaining understandable to users who might not be familiar with software implementation. It is also flexible enough to support scenario comparison, policy testing, and future changes to the curriculum or admissions pattern. In this sense, the simulator is not only a technical artifact; it is also a decision-support tool for educational planning.

To strengthen the technical evaluation of the system, the implementation was reviewed through both validation and output evidence. The project includes automated tests for determinism, prerequisite logic, graduation behavior, capacity allocation, API behavior, and database behavior. It also produces structured outputs such as `outputs/reports/simulation_summary.csv`, `outputs/reports/flow_timeline.json`, and the figures stored in `outputs/figures/`, which demonstrate how the simulation results can be inspected and compared in practice, and which are the source of the results reported later in this section.

Understanding the Domain and Building the Core Engine

I began by understanding the curriculum and designing the core simulation approach. I reviewed the Computer Science study plan, prerequisite structure, and the main assumptions needed to represent student movement through the program.

I then built the core Python simulation engine. The engine uses a discrete-term loop where each student attempts to register for courses, receives seats based on capacity and priority, and then either passes or fails enrolled courses. Each student has their own academic state, including completed courses, credit hours, GPA, probation status, and graduation status. I also extended this early version from a single-class model into a multi-cohort one by adding a GPA-driven dropout rule and a shared capacity bottleneck across course sections, which set the direction for the rest of the project.

One important design decision was to model students individually instead of using only aggregate cohort totals. This was necessary because prerequisite eligibility depends on each student's personal course history. A student who failed a gateway course may be blocked from several later courses even if the rest of the cohort is progressing normally.

From Script to Full-Stack Application

I then moved the project from a script-based simulation into a full-stack application. I first ported the static output figures and the prerequisite-network diagram into an animated, interactive Next.js and React dashboard, replacing a set of one-off image files with pages a user could actually click through. I then built out the backend: a FastAPI layer that ran the simulation through HTTP endpoints, and a SQLAlchemy/SQLite persistence model so that courses, plans, configurations, scenarios, and runs could be represented in a structured way, alongside a first multi-page dashboard and a Scenario Builder.

I also began work on the multi-plan model, so that more than one curriculum-and-configuration combination could be stored, imported, and compared without one overwriting another, together with a Plan Builder wizard for creating a new plan from scratch or by cloning the default one, plus a first visual design pass on the dashboard.

A First Live Simulation Mode, and Learning to Cut Scope

I built the first version of a stepwise, term-by-term simulation mode, an early "Live Simulation" feature, so a user could watch the model advance one term at a time instead of only seeing a finished run. I also added a Bottlenecks page with capacity-section recommendations, ranking courses by which one would benefit most from additional seats.

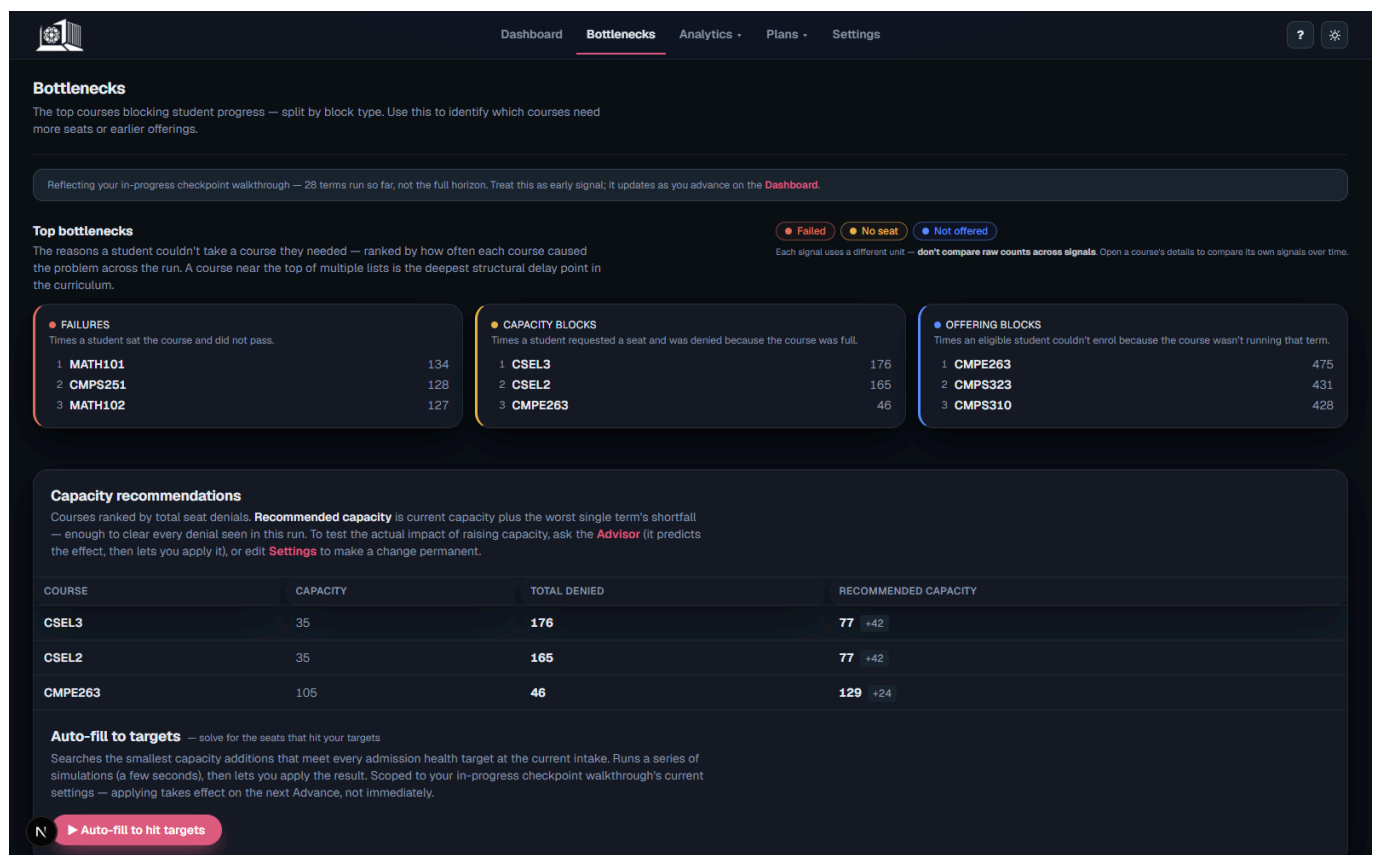
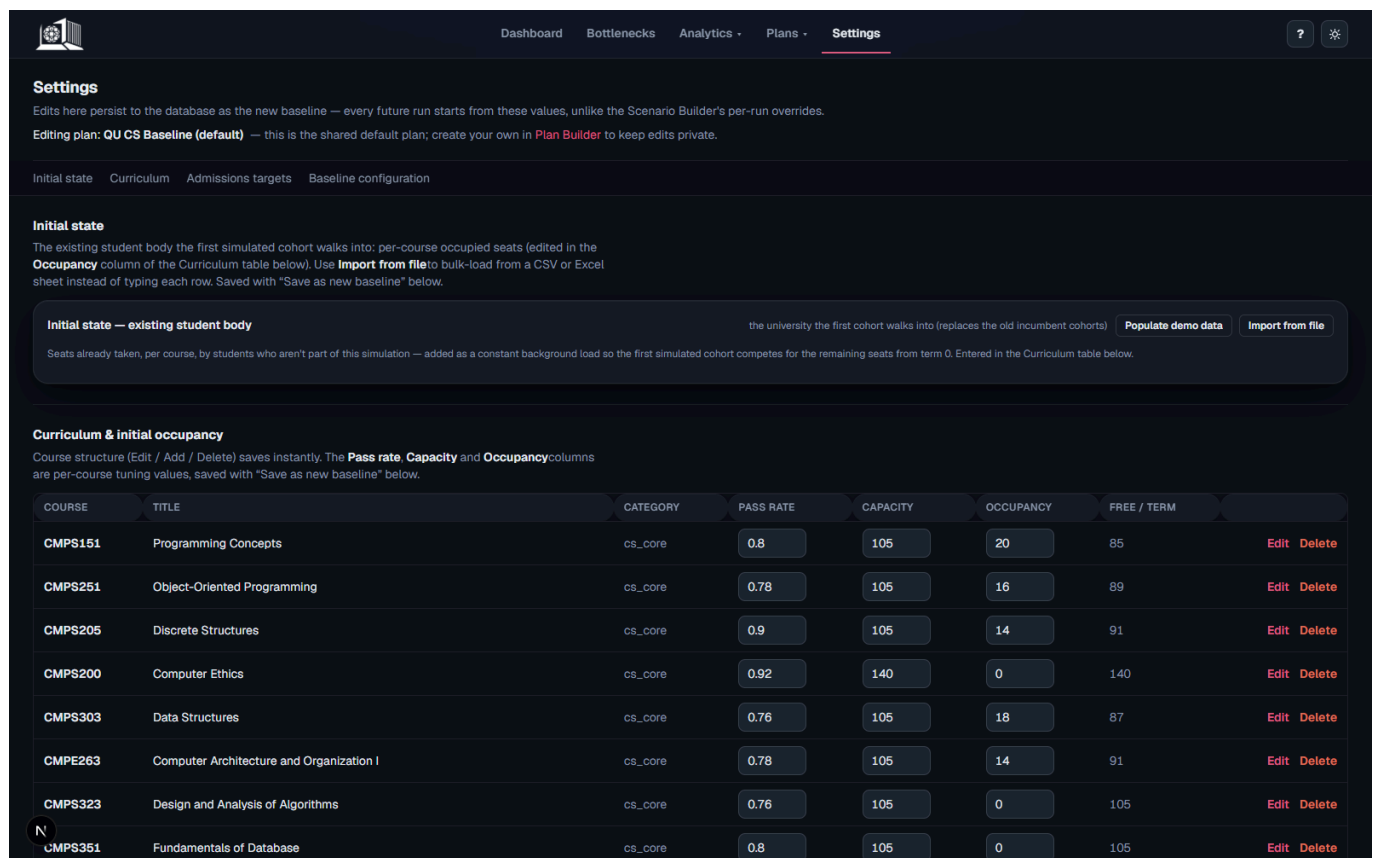


Figure 1 presents the bottlenecks analysis page, where the system ranks the main causes of delay, failure, capacity pressure, and offering limitations, separately, and suggests a capacity target for each flagged course.

Soon after, I made a deliberate simplification pass: I removed authentication, the Scenario Builder, a separate Capacity Planning page, and an instructor concept that had accumulated but were not central to the research question, in favor of a single shared demo user and a leaner what-if panel merged directly into Bottlenecks. This was the first of several points in the internship where building a feature and then honestly re-evaluating whether it served the project's actual question turned out to be more valuable than keeping everything I had built (discussed further under Challenges Faced below).

Initial-State Modeling and the First Decision-Support Tools

I then introduced the Initial-State model, which replaced an earlier simulated-incumbents warm start. Instead of simulating a second, fictitious population of already-enrolled students, the university's actual current occupancy is entered directly: a required first-run setup gate asks for how many seats in each course are already taken by the existing student body, with a CSV bulk-import option for entering many courses at once.



Settings

Edits here persist to the database as the new baseline — every future run starts from these values, unlike the Scenario Builder's per-run overrides.

Editing plan: **QU CS Baseline (default)** — this is the shared default plan; create your own in **Plan Builder** to keep edits private.

Initial state Curriculum Admissions targets Baseline configuration

Initial state

The existing student body the first simulated cohort walks into: per-course occupied seats (edited in the **Occupancy** column of the Curriculum table below). Use **Import from file** to bulk-load from a CSV or Excel sheet instead of typing each row. Saved with "Save as new baseline" below.

Initial state — existing student body the university the first cohort walks into (replaces the old incumbent cohorts) **Populate demo data** **Import from file**

Seats already taken, per course, by students who aren't part of this simulation — added as a constant background load so the first simulated cohort competes for the remaining seats from term 0. Entered in the Curriculum table below.

Curriculum & initial occupancy

Course structure (Edit / Add / Delete) saves instantly. The **Pass rate**, **Capacity** and **Occupancy** columns are per-course tuning values, saved with "Save as new baseline" below.

COURSE	TITLE	CATEGORY	PASS RATE	CAPACITY	OCCUPANCY	FREE / TERM	
CMPS151	Programming Concepts	CS_core	0.8	105	20	85	Edit Delete
CMPS251	Object-Oriented Programming	CS_core	0.78	105	16	89	Edit Delete
CMPS205	Discrete Structures	CS_core	0.9	105	14	91	Edit Delete
CMPS200	Computer Ethics	CS_core	0.92	140	0	140	Edit Delete
CMPS303	Data Structures	CS_core	0.76	105	18	87	Edit Delete
CMPE263	Computer Architecture and Organization I	CS_core	0.78	105	14	91	Edit Delete
CMPS323	Design and Analysis of Algorithms	CS_core	0.76	105	0	105	Edit Delete
CMPS351	Fundamentals of Database	CS_core	0.8	105	0	105	Edit Delete

Figure 2 shows the settings page, where curriculum and configuration values, including this initial occupancy, can be adjusted. It highlights the flexibility of the platform and shows that the simulation can be adapted to different planning scenarios and institutional policies.

I also built the first version of two decision-support features on top of a completed run: a rules-based Advisor panel that reads a run's health criteria and top bottlenecks into a prioritized, plain-language list of recommendations, and a bounded Auto-fill solver that searches for the smallest capacity increases needed to meet an admissions target. The Advisor was given its own dedicated page in the dashboard for the first time.

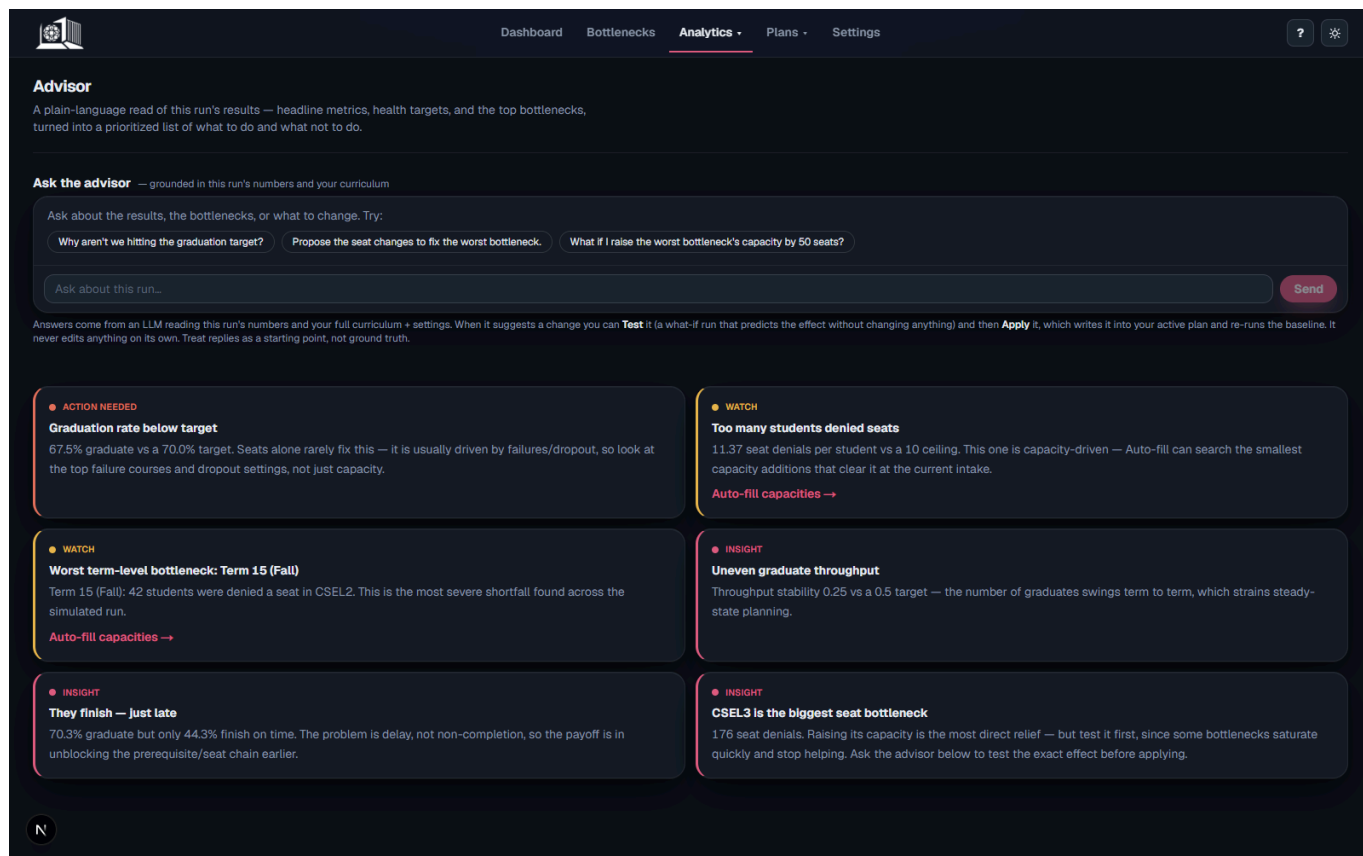


Figure 3 illustrates the advisor page in its current form. At this stage in the internship it held only the rules-based recommendations described above; the grounded chat interface and testable proposals shown here were added later.

Realistic Scheduling, an LLM-Grounded Advisor, and Performance

This was one of the busiest stretches of the internship. I reworked the course-offering schedule to match Qatar University's real pattern instead of a blanket Fall-and-Spring assumption: several core courses run in only one term a year, and non-CS service courses additionally run in an optional Summer term. Alongside that, I switched admissions from a simulated incumbent population to the steady-state model described in Section 1, with several overlapping cohorts admitted every year into one shared pool of seats.



Figure 4 shows the analytics view, where university enrollment trends and per-cohort flow patterns are displayed. These charts only became a meaningful signal once the offering schedule and admissions model behind them were realistic; before that, they mostly showed an artifact of the simplified assumptions rather than a pattern worth planning around.

I also upgraded the Advisor from the rules-based panel built earlier into an optional LLM-backed chat, grounded in the run's numbers and the full active curriculum, that can answer detailed course-level questions and, when it recommends a concrete change, propose it as a card the user can first test in a sandboxed simulation and only then apply; the model itself never writes to the plan directly. Alongside these modeling changes, I did a round of performance work, parallelizing the Monte Carlo and Auto-fill workloads across processes and caching a monotonic eligibility check, and a long pass of dashboard layout and accessibility polish.

Replacing Live Simulation with a Resumable Semester Checkpoint Mode

It became clear that the earlier Live Simulation feature did not match how an academic planner actually works: its continuous-tick, replay-log design let a user watch a run happen, but not cleanly pause, edit a future setting such as next year's intake or a course's capacity, and continue from exactly that point. I removed it, from both the frontend and the backend, and replaced it with Semester Checkpoint Mode.

Building the replacement meant first making the simulator itself resumable: a new method advances the engine by exactly one calendar term, and the engine can snapshot and later restore its full internal state, so a session can pause after one request and resume in a brand-new process on the next. I exposed that as a session-based API and a dedicated dashboard page, then, once it was working end to end, merged the checkpoint walkthrough into being the dashboard itself rather than a separate page, since that was the page a planner would actually want to land on first.

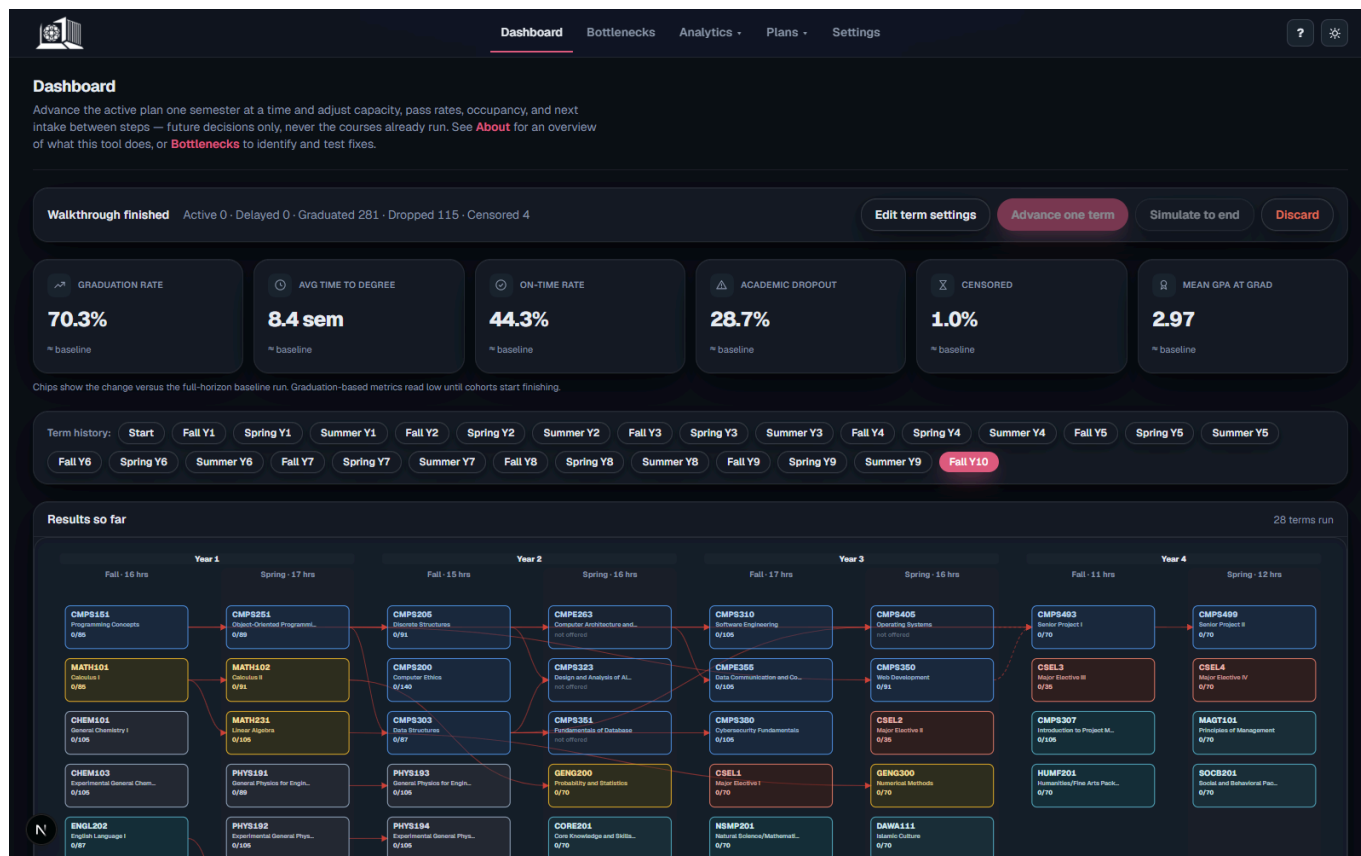


Figure 5 shows the dashboard in its current, checkpoint-driven form, where a user reviews headline results next to a program-roadmap layout of the curriculum and advances the walkthrough one term at a time. It is the page every planning session now starts from.

Around the same time, I also added central plan-edit guardrails, locked prerequisite editing to course creation only so a mid-life edit could not silently invalidate a run built on the old prerequisite graph, fixed a real import error I had introduced while wiring the new guardrails in, and added retake-cap enforcement and a severe-terms forecast summary.

Rewind, Final Polish, and a Second Round of Scope Discipline

Toward the end of the internship, I added the ability to go back to a previous term inside Checkpoint Mode, then split that into a free, read-only preview step and a separate, explicit "Continue from here" commit, so looking at an earlier term and deciding to act on it are no longer the same click. I also made Checkpoint Mode pause at every calendar term, including the optional Summer and Winter ones, not only Fall and Spring.

I removed the Per-Student Trace feature I had built earlier in the internship, once it became clear it was not adding to the project's core planning question even though it worked correctly, the same scope discipline described below under Challenges Faced, applied a second time to my own later work rather than only to the earlier features I had inherited from the project's early direction.

To close out the internship, I added a pinned KPI strip with baseline deltas and a "Simulate to end" shortcut to the checkpoint dashboard, added automated syntax and lint checks to my own development workflow, improved the test suite, and finished the documentation, including the results reported later in this section and this report itself.

Challenges Faced

Modeling Optional Terms Correctly

One major challenge, which surfaced while reworking the offering schedule to be realistic, was supporting optional academic terms such as Winter or Summer. The original logic assumed that every term counted toward a student's academic time budget. This became incorrect when optional terms were introduced, because students should be able to take an optional catch-up term without necessarily using one of their regular semesters.

The solution was to separate global simulation time from each student's personal semester count. Mandatory terms advance the student's personal semester count, while optional terms can still affect course outcomes without consuming the same academic time budget. This made the model more realistic and prevented students from being cut off too early.

Maintaining Determinism

Another challenge, decided at the very start of the internship and never revisited, was ensuring that the simulation remained deterministic. Since the model includes random pass and fail outcomes, repeated runs could become difficult to compare if randomness changed unpredictably. To solve this, each simulated student uses a seeded random stream based on the global seed and student ID. This means the same student receives the same random sequence when comparing scenarios.

This design is important for what-if analysis. If a capacity change improves the results, the difference should come from the capacity change itself, not from a different random draw. It is also what makes the close agreement between the single run and the Monte Carlo average, shown below in Results and Findings, a meaningful check rather than a coincidence.

Balancing Features and Focus

As the project grew, it became clear that adding more features did not always make the system better. Some features increased complexity without directly supporting the main research question, and a few (the original Live Simulation mode, and later the Per-Student Trace) were built, worked correctly, and were still removed once it was clear they were not earning their place. I learned to simplify the application and focus on the capabilities that mattered most: simulation, bottleneck analysis, scenario comparison, configuration, and clear visualization.

This was an important professional lesson. Good software is not only about adding functionality; it is also about choosing the right scope and keeping the system understandable, and being willing to cut something I had already built once it stopped earning its place.

Results and Findings

Running the full simulation on the current baseline configuration, an eight-cohort steady-state university of 100 students admitted every Fall for eight years, produces the headline outcomes summarized in Table 1. A single deterministic run and a 30-seed Monte Carlo average agree closely, which is expected given the model's seeded random-number design (discussed above under Challenges Faced), and is itself a useful check that the reported figures are not an artifact of one lucky or unlucky seed.

Table 1. Baseline headline metrics (single run vs. 30-seed Monte Carlo average)

Metric	Single run	Monte Carlo mean (95% CI)
Graduation rate	66.1%	65.8% (65.4%-66.2%)

Metric	Single run	Monte Carlo mean (95% CI)
Academic dropout rate	30.5%	30.9% (30.4%-31.3%)
Censored rate (hits the 12-semester horizon)	3.4%	3.4% (3.0%-3.7%)
Average time to graduate	8.8 semesters	8.8 semesters (8.73-8.77)
On-time rate (graduates within 8 semesters)	33.5%	33.5% (32.9%-34.0%)
Probation rate	14.2%	13.7% (13.4%-14.0%)
Mean GPA at graduation	2.97	2.97

Two-thirds of students graduate within the 12-semester horizon, but only about a third graduate on the nominal 8-semester schedule, and the average graduating student needs almost nine semesters rather than eight. Reading the four bottleneck signals separately, rather than as one combined score, points to two distinct causes behind that gap rather than one:

- **Failure is concentrated in the mathematics gateway sequence.** MATH102, MATH101, and MATH231 are the three most-failed courses in the run, each with well over 200 failed attempts across the simulated population. These are early courses that gate a large share of the downstream curriculum, so a failure here has an outsized effect on a student's whole trajectory.
- **Capacity pressure is concentrated in major electives, by design.** CSEL2 and CSEL3 account for by far the largest number of capacity-blocked requests in the run (907 and 880 respectively), well ahead of any other course. This is the intended effect of the sizing policy described above under Analytics and Decision Support: electives are deliberately squeezed to keep a controlled bottleneck in a place where four interchangeable slots and no prerequisite chain make the cost of contention lowest. The offering signal, in contrast, is dominated by the handful of core courses that Qatar University schedules in only one term a year (CMPE263, CMPS310, CMPS323), which shows up as a distinct, schedule-driven form of delay rather than a seat-shortage one.
- **Prerequisite waiting is heaviest just before the senior project.** CMPS499 and CMPS493 show the largest prerequisite-block counts by a wide margin, which is consistent with them sitting at the end of a long compound prerequisite chain rather than being a bottleneck in their own right; the tool's own documentation flags this signal as a passive, whole-curriculum sweep and treats it as the least actionable of the four for exactly that reason.

The admissions-recommendation module, which reads the same slack-based health criteria as the advisor panel, flagged seat-denial pressure as the binding constraint on this configuration and suggested a substantially smaller yearly intake than the current 100 students under the admission targets currently configured for the plan. Taken literally this number is far too aggressive to act on; what it usefully demonstrates is that the model's recommendation logic is sensitive to exactly the elective-capacity pressure identified above, and that intake size and elective capacity are two levers acting on the same underlying constraint. In practice, raising CSEL2/CSEL3 capacity (or applying the Auto-fill solver described earlier in this section) is a more targeted response than reducing intake, since it relieves the actual bottleneck instead of shrinking the whole cohort around it.

Together, these results support the project's central premise: a single graduation-rate number would have shown "roughly a third of students are delayed past the nominal schedule" without saying why. Separating the four signals shows that the delay in this curriculum is driven mainly by failure risk concentrated in a few math

courses and by a deliberately scarce elective pool, with single-term scheduling as a secondary, structurally distinct contributor, each of which points to a different, concrete intervention.

4. Reflections and learning outcomes

Overall, the internship met the expectations I set for myself at the start: I wanted hands-on, end-to-end ownership of a real software system rather than a short exercise, and the self-directed structure of the Scale x Qatar University Practical Training program gave me exactly that. It also sharpened my sense of the kind of engineering work I want to keep doing after graduation: building systems where the correctness of the underlying model matters as much as the code that implements it, and where results have to be explained clearly to people who were not in the room while the system was built.

This internship helped me improve both my technical and professional skills. Technically, I gained experience in simulation design, backend API development, database schema design, frontend dashboards, analytics, and automated testing. I also learned how to structure a project so that the same simulation engine can be used by a command-line script, an API, and a web dashboard without duplicating logic.

The project also strengthened my understanding of concepts from my coursework. Data structures and algorithms were used in representing prerequisite relationships and validating curriculum graphs. Database systems were used in designing the plan-scoped persistence model. Software engineering principles were used in separating the engine, service layer, API, database, and frontend. Probability and statistics were used in pass/fail modeling, reproducible randomness, and the Monte Carlo analysis reported under Internship Experience. Several specific university courses and concepts were directly applied during this internship: in Data Structures and Algorithms, I used graph-based reasoning to represent prerequisite chains and validate curriculum topology; in Database Systems, I designed the persistence model using SQLAlchemy and SQLite to manage plans, courses, configurations, and run data; in Software Engineering, I applied modular design, testing, and separation of concerns while building a multi-layer application; in Probability and Statistics, I worked with seeded randomness and scenario comparison to make simulation outcomes reproducible and meaningful; and in Web Development, I used React, TypeScript, and Next.js to transform simulation results into an interactive and understandable dashboard. The main difference from the classroom was that the work had to be reliable, explainable, and useful for real academic planning decisions rather than only solving a small standalone assignment.

Professionally, I learned the value of communicating technical results clearly. A simulation can produce many numbers, but users need to understand what those numbers mean and what action they suggest. The dashboard, figures, and bottleneck categories helped turn technical output into information that could support discussion, and writing the Results and Findings part of this report was itself a useful exercise in doing that in prose rather than only in the dashboard's own visual language. Working with a remote host organization while based on Qatar University's campus also pushed me to communicate progress in writing more than I would have needed to if a Scale AI mentor had been down the hall, which is its own skill I expect to keep using. I also learned that documentation is part of engineering work, not something separate from it: writing project overviews, technical design notes, API references, and assumptions helped keep the project consistent and made it easier to explain decisions later.

Looking beyond the internship itself, the project also pointed me toward what I would want to keep developing if I continued working on it: calibrating the simulator against real historical enrollment and graduation data rather than assumed pass rates, building richer side-by-side scenario comparisons for policy interventions, and extending the advisor's guided suggestions further. The most concrete next step the model

itself points to is raising CSEL2/CSEL3 capacity toward the elective sizing policy's own peak-demand standard, or running the Auto-fill solver against a seat-denial target, to check whether that alone closes most of the gap between the 66% horizon graduation rate and the 33% on-time rate, before considering any change to intake size. Wanting to keep pushing on a system after the internship officially ended is, to me, a good sign that the objectives I set in Section 2 were the right ones.

The project achieved its main objective by modeling curriculum progression and identifying the causes of delay through separate bottleneck signals, a concrete, evidence-based answer to the question I set out to ask when I chose this problem for my catalog task. Overall, this internship improved my confidence as a developer and showed me the importance of building systems that are not only technically functional, but also explainable, focused, and useful to their intended audience.